

Question 1: What is the difference between list and tuple in Python?

- A) List is mutable, Tuple is immutable
- B) List is immutable, Tuple is mutable
- C) List holds homogeneous data, Tuple holds heterogeneous data
- D) There is no difference

Correct Answer: A

Explanation: Lists are mutable and can be modified after creation. Tuples are immutable and their contents cannot be changed.

Difficulty: easy

Topic: data-structures

Question 2: Which of the following creates a dictionary in Python?

- A) `x = []`
- B) `x = ()`
- C) `x = {}`
- D) `x = set()`

Correct Answer: C

Explanation: Curly braces `{}` are used to define a dictionary in Python. `x = []` creates a list, `x = ()` creates a tuple, and `x = set()` creates an empty set.

Difficulty: easy

Topic: data-structures

Question 3: What does the 'is' keyword evaluate in Python?

- A) Equality of values
- B) Identity/memory address equality
- C) Type equality
- D) Subclass relationships

Correct Answer: B

Explanation: The 'is' operator checks if two reference variables point to the exact same object in memory, whereas '==' checks if the values of two objects are equal.

Difficulty: easy

Topic: operators

Question 4: What does `print(type([]))` output?

- A) `<class 'list'>`
- B) list
- C) []
- D) Error

Correct Answer: A

Explanation: In Python 3, all data types are classes. Evaluating type of a list literal returns the list class.

Difficulty: easy

Topic: data-structures

Question 5: How do you reverse a list 'my_list' using slicing?

- A) `my_list[::-]`
- B) `my_list[::-1]`
- C) `my_list[-1::]`
- D) `my_list[1::-1]`

Correct Answer: B

Explanation: The slicing format is start:stop:step. A step of -1 traverses the list in reverse order.

Difficulty: easy

Topic: slicing

Question 6: What happens when a default mutable argument is modified in a function?

- A) It is reset to empty on every function call
- B) It retains modifications across calls because default arguments are evaluated once at definition time
- C) A `TypeError` is raised
- D) A new copy is created automatically

Correct Answer: B

Explanation: Python's default arguments are evaluated once when the function is defined. If you use a mutable default parameter (like a list) and modify it, the changes persist across subsequent function calls.

Difficulty: medium

Topic: functions

Question 7: Which types can be used as keys in a Python dictionary?

- A) Only strings
- B) Any mutable type
- C) Any hashable (immutable) type
- D) Any type including lists and dictionaries

Correct Answer: C

Explanation: Dictionary keys must be hashable, meaning they must have a hash value that never changes during their lifetime. This includes immutable types like strings, numbers, and tuples (if all their elements are hashable).

Difficulty: medium

Topic: data-structures

Question 8: What is the primary difference between a generator (using `yield`) and a regular function?

- A) Generators are faster but use more memory
- B) Generators return an iterator that yields values lazily, preserving state between yields, whereas functions return a single value and terminate
- C) Generators can only yield strings
- D) Functions can be recursive but generators cannot

Correct Answer: B

Explanation: Generators generate values on-the-fly (lazy evaluation) and pause their state, saving memory compared to returning a full list from a function.

Difficulty: medium

Topic: generators

Question 9: What is the purpose of the `functools.wraps` decorator when writing a custom decorator?

- A) To speed up execution of the decorated function
- B) To preserve the original function's metadata (name, docstring, etc.)
- C) To allow the decorator to accept arguments
- D) To enforce type safety

Correct Answer: B

Explanation: Without `functools.wraps`, the decorated function inherits the metadata of the wrapper function. `wraps` copies the `__name__`, `__doc__`, and other attributes back to the original function.

Difficulty: medium

Topic: decorators

Question 10: What is the output of `print(bool('False'))` in Python?

- A) `False`

- B) True
- C) None
- D) Error

Correct Answer: B

Explanation: Any non-empty string in Python evaluates to True when cast to a boolean. Only empty strings ("") evaluate to False.

Difficulty: medium

Topic: types

Question 11: Which of the following is correct regarding Python's Global Interpreter Lock (GIL)?

- A) It allows multiple threads to run Python bytecode in parallel on multiple CPU cores
- B) It limits execution to a single thread at a time for Python bytecode, preventing CPU-bound parallel execution in CPython
- C) It speeds up I/O bound multi-threaded operations
- D) It is present in all Python implementations including Jython and IronPython

Correct Answer: B

Explanation: The GIL is a mutex that protects access to Python objects, preventing multiple threads from executing Python bytecodes at once. It is specific to the reference implementation CPython.

Difficulty: hard

Topic: concurrency

Question 12: In Python's model, what is the difference between `__new__` and `__init__`?

- A) `__init__` is the constructor, `__new__` is the destructor
- B) `__new__` is a static method responsible for creating and returning a new instance, while `__init__` is an instance method responsible for initializing it
- C) `__new__` is called after `__init__`
- D) `__new__` only works for subclasses

Correct Answer: B

Explanation: `__new__` is the actual creator of the object, whereas `__init__` is the initializer. `__new__` takes the class reference (cls) as its first argument and returns the instance, which is then passed to `__init__`.

Difficulty: hard

Topic: oop

Question 13: What is the purpose of Method Resolution Order (MRO) in Python multiple inheritance?

- A) To optimize database queries in Django
- B) To define the search order Python uses to lookup methods/attributes on an object in multiple inheritance, resolved via the C3 linearization algorithm
- C) To sort class attributes alphabetically
- D) To resolve cyclic imports

Correct Answer: B

Explanation: MRO defines the order in which base classes are searched when looking up a method or attribute. Python uses the C3 linearization algorithm to construct this order, avoiding the Diamond Problem.

Difficulty: hard

Topic: oop

Question 14: What will be the output of `print([x for x in range(3)] == (x for x in range(3)))`?

- A) True
- B) False
- C) Error
- D) None

Correct Answer: B

Explanation: The left side is a list comprehension which produces a list `([0, 1, 2])`. The right side is a generator expression which produces a generator object. A list and a generator are not equal, even if they yield the same elements.

Difficulty: hard

Topic: comprehensions

Question 15: How does python manage memory and clean up unused objects?

- A) Solely through manual memory management like malloc/free
- B) Using reference counting as the primary mechanism, supplemented by a cyclic garbage collector to detect and clean reference cycles
- C) Solely through a stop-the-world mark-and-sweep garbage collector
- D) Python does not clean up memory until the process terminates

Correct Answer: B

Explanation: CPython uses reference counting. When an object's reference count drops to 0, it is immediately deallocated. However, because self-referencing loops (reference cycles) would never drop to 0, Python uses a generational cyclic garbage collector to find and clean these cycles.

Difficulty: hard

Topic: memory

Question 16: What is the difference between `copy.copy()` and `copy.deepcopy()` in Python?

- A) `copy.copy()` copies lists, `copy.deepcopy()` copies dictionaries
- B) `copy.copy()` performs a shallow copy (copies references to nested objects), while `copy.deepcopy()` recursively duplicates all nested objects
- C) `copy.deepcopy()` runs in $O(1)$ time
- D) There is no difference

Correct Answer: B

Explanation: Shallow copy creates a new compound object but inserts references into it to the objects found in the original. Deep copy creates a new compound object and recursively inserts copies of the objects found in the original.

Difficulty: medium

Topic: data-structures

Question 17: What is name mangling in Python classes?

- A) A runtime optimization that obfuscates function names to run faster
- B) Prefixing an attribute with two leading underscores (e.g. `__member`) to rewrite its name internally as `_ClassName__member` to prevent naming collisions in subclasses
- C) Overloading class constructors
- D) Replacing variable names with numbers

Correct Answer: B

Explanation: Name mangling is used to protect class-private variables from being overridden by subclasses. Any identifier of the form `__spam` is textually replaced with `_classname__spam`.

Difficulty: medium

Topic: oop

Question 18: What is the output of `print(1 or 2, 1 and 2)`?

- A) True, False
- B) 1, 2
- C) 1, 1
- D) 2, 1

Correct Answer: B

Explanation: Python's logical operators 'or' and 'and' evaluate lazily and return the actual operands, not boolean values. '1 or 2' evaluates 1 (truthy) and stops, returning 1. '1 and 2' evaluates 1 (truthy), then evaluates 2, returning 2.

Difficulty: medium

Topic: operators

Question 19: Which magic method implements length checking (len()) in custom classes?

- A) `__size__`
- B) `__length__`
- C) `__len__`
- D) `__count__`

Correct Answer: C

Explanation: To define how the built-in len() function behaves on your custom class, implement the `__len__(self)` magic method.

Difficulty: easy

Topic: oop

Question 20: What happens if an exception is raised inside a try block and there is a finally block?

- A) The finally block is skipped if the exception is unhandled
- B) The finally block is executed always, before control leaves the try/except/finally structure
- C) The program crashes immediately before executing finally
- D) finally is only executed if no exception was raised

Correct Answer: B

Explanation: The finally block is guaranteed to execute whether an exception is raised or not, and whether it is caught or not. If finally returns or raises a new exception, it overrides the original exception.

Difficulty: medium

Topic: exceptions